

Plan (short)

1. Install Apache and ModSecurity (libapache2-mod-security2).
2. Enable ModSecurity in **DetectionOnly** mode and install OWASP CRS (Core Rule Set).
3. Test detection with safe test payloads, inspect ModSecurity audit logs.
4. Tune rules (disable false positives / add custom rules).
5. Switch to blocking (SecRuleEngine On) when tuned.
6. Collect outputs .

Step-by-step (Apache on Debian / Kali / Ubuntu)

1) Update packages & install Apache + ModSecurity

Run:

```
sudo apt update
```

```
sudo apt install -y apache2 git
```

```
sudo apt install -y libapache2-mod-security2
```

Check apache & module:

```
sudo systemctl status apache2
```

```
sudo apache2ctl -M | grep security2 # should show security2_module
```

2) Activate the default ModSecurity configuration (use recommended file)

The package usually supplies a recommended config. Copy it and set detection mode:

```
sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
```

Ensure DetectionOnly initially:

```
sudo sed -i "s/SecRuleEngine .*/SecRuleEngine DetectionOnly/" /etc/modsecurity/modsecurity.conf
```

You can confirm:

```
grep -i "^SecRuleEngine" /etc/modsecurity/modsecurity.conf
```

(DetectionOnly means ModSecurity will log findings but not block — good for tuning.)

3) Install OWASP Core Rule Set (CRS)

CRS gives a ready set of rules:

```
sudo mkdir -p /usr/share/modsecurity-crs
```

```
sudo git clone https://github.com/coreruleset/coreruleset /usr/share/modsecurity-crs
```

```
cd /usr/share/modsecurity-crs
```

```
sudo cp crs-setup.conf.example crs-setup.conf
```

Include CRS in Apache ModSecurity config. Edit the security2 conf:

```
sudo nano /etc/apache2/mods-enabled/security2.conf
```

Add (**near top**):

```
# OWASP CRS
```

```
IncludeOptional /usr/share/modsecurity-crs/crs-setup.conf
```

```
IncludeOptional /usr/share/modsecurity-crs/rules/*.conf
```

Save & exit.

Restart Apache:

```
sudo systemctl restart apache2
```

4) Verify ModSecurity + CRS is active

```
sudo apache2ctl -M | grep security2
```

```
# Verify CRS files present
```

```
ls -l /usr/share/modsecurity-crs/rules | head
```

5) Do a simple detection test

From the host (or a machine allowed to hit your webserver) run a test request that CRS flags (XSS example):

```
curl -i 'http://localhost/?q=<script>alert(1)</script>'
```

You should still get a normal response because we're in DetectionOnly mode. Next check the ModSecurity audit log:

Common audit/log locations:

- /var/log/modsec_audit.log (if configured by your package)

- `/var/log/apache2/error.log` (ModSecurity messages often appear here)

Check the last lines:

```
sudo tail -n 200 /var/log/modsec_audit.log
```

```
sudo tail -n 200 /var/log/apache2/error.log | grep -i "modsecurity\|CRS"
```

If detection worked you'll see an entry describing the rule triggered, rule id, and request URI.

6) Interpret the log entry (what to look for)

In the ModSecurity audit event you'll typically see:

- **Unique transaction ID**
- **Client IP** and request line (GET /?q=...)
- **Which rule(s) triggered** (rule ID like 941100 etc.)
- **Action** (usually Warning in DetectionOnly)
- **Matched data** snippet

7) Tuning — handle false positives

When rules cause false positives you can:

- Disable a specific rule id: `SecRuleRemoveById <id>`
- Exclude targets for a rule: `SecRuleUpdateTargetById <id> !REQUEST_HEADERS:Referer`
- Use `crs-setup.conf` to adjust paranoia level / excluded rules.

Create a file for your local rule overrides:

```
sudo nano /etc/modsecurity/custom_exceptions.conf
```

Example to remove a rule:

```
# disable problematic rule temporarily
```

```
SecRuleRemoveById 981176
```

Include this file from `/etc/modsecurity/modsecurity.conf` (add `Include /etc/modsecurity/custom_exceptions.conf`).

After any change restart apache:

```
sudo systemctl restart apache2
```

8) Example custom rule(s)

Add an easy custom rule to block a suspicious user agent (for lab testing):

```
sudo tee /etc/modsecurity/local_rules.conf <<'EOF'
```

```
# Block agents containing "bad-scanner"
```

```
SecRule REQUEST_HEADERS:User-Agent "bad-scanner" "id:1000001,phase:1,deny,log,msg:'Blocked  
bad user agent from lab test'"
```

```
EOF
```

Include the file in /etc/modsecurity/modsecurity.conf:

```
Include /etc/modsecurity/local_rules.conf
```

Restart Apache.

Test by curling with that UA:

```
curl -i -A "bad-scanner" http://localhost/
```

```
# If SecRuleEngine is DetectionOnly you will see logs but still get content. Switch to On to block.
```

9) Enable blocking (only after tuning)

When you are satisfied (few false positives), change:

```
sudo sed -i 's/SecRuleEngine DetectionOnly/SecRuleEngine On/' /etc/modsecurity/modsecurity.conf
```

```
sudo systemctl restart apache2
```

Now known bad requests will be blocked (403 or custom response).
